

PRODUCT SALES ANALYTICS

END-TO-END PRODUCT SALES ANALYTICS PROJECT

NAME: R. KAMALENDRAN

Git repo link: <https://github.com/kamalendran464-byte/Sales-Analysis-Report-.git>

Project Overview

The objective of this project is to develop an end-to-end Sales Analytics solution that helps a business understand its sales performance, customer behavior, product performance, regional sales, profitability, and overall business growth.

The project covers the complete analytics lifecycle, starting from raw sales data and ending with an interactive Power BI dashboard and actionable business recommendations.

Tools and Technologies

The following tools will be used:

- Google Co-Lab – Data exploration and initial cleaning, Data extraction, transformation, and analysis
- SQL – performing some query
- Power BI – Dashboard development
- GitHub – Project documentation and portfolio

Dataset structure

Your Excel file contains **4 tables**:

Table	Rows	Columns	Purpose
Sales Orders	7,991	12	Main transaction/fact table
Customers	50	2	Customer master
Regions	100	7	Delivery/location master
Products	15	2	Product master

Data Model

Table	Role	Important Fields
Sales Data	Fact / transaction table	Order Number, Order Date, Quantity, Unit Selling Price, Unit Cost, Sales, Profit, Channel
Customer Data	Customer dimension	Customer Index, Customer Names
Products Data	Product dimension	Product Index, Product Name
Regions Table	Geography dimension	Index, City, Suburb, postcode, Address, Latitude, Longitude
DateTable	Date dimension	Date, Year, Month, Month Sort, Month-Year
Measure Table	Measures	Sales, Sales PY, Sales vs PY, Profit, Profit LY, Profit Margin

Data Relationship

The datasets were connected using common identifiers such as Sales Orders Customer Name Index ,Order Number , Order Date ,Ship Date, source and hiring-outcome information to be combined for end-to-end recruitment analysis.

Integrated Dataset

The final integrated dataset contains **7,991 sales transaction-level records and 31 columns**. The dataset combines information from **sales orders, customers, products, and regions** into a single analytical structure. Derived features were also created to support **sales, profitability, customer, product, and regional performance analysis**.

Data Area	Representative Fields
Sales Order	Order Number, Order Date, Ship Date, Channel, Currency Code, Warehouse Code, Order Quantity, Unit Selling Price, Unit Cost
Customer	Customer Index, Customer Name Index, Customer Names
Product	Product Description Index, Product Name
Region	Delivery Region Index, Suburb, City, postcode, Longitude, Latitude, Full Address

Sales Metrics	Sales Amount, Cost Amount, Profit, Profit Margin %
Time Features	Order Year, Order Month, Order Month Name, Order Quarter
Derived Features	Sales Amount = Order Quantity × Unit Selling Price, Cost Amount = Order Quantity × Unit Cost, Profit = Sales Amount – Cost Amount, Profit Margin %

Dataset Summary

- **Records:** 7,991
- **Columns:** 31
- **Granularity:** Individual sales order/transaction
- **Main Fact:** Sales Orders
- **Dimension Areas:** Customer, Product, Region, Date
- **Analysis Areas:** Sales, Cost, Profit, Margin, Quantity, Customer, Product, Region, Channel, and Time

. Key Business Measures

Measure	Purpose
Sales	Measures revenue generated from sales transactions.
Sales PY	Compares current sales with the previous-year sales value.
Sales vs PY	Shows the variance or comparison between current sales and previous-year sales.
Profit	Measures sales value after deducting the associated cost.
Profit LY	Provides previous-year profit for historical comparison.
Profit Margin	Shows profitability relative to sales.

Google co-lab

Step 1: Analysis the data using with google co-lab

Upload the dataset to Google Co-lab (xlsx or csv)

```
uploaded = files.upload()
```

Select: Integrated_Sales_Dataset.xlsx

Read the data :

```
df = pd.read_excel('Sales Analysis Report.xlsx')
```

show the data :

```
df.head()
```

<

Display all column names

```
df.columns()
```

```
df.shape
Out[1]: (7991, 12)

df.info()
Out[1]:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7991 entries, 0 to 7990
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  --
0   OrderNumber            7991 non-null   object
1   OrderDate              7991 non-null   datetime64[ns]
2   Ship Date              7991 non-null   datetime64[ns]
3   Customer Name Index    7991 non-null   int64
4   Channel                7991 non-null   object
5   Currency Code          7991 non-null   object
6   Warehouse Code         7991 non-null   object
7   Delivery Region Index  7991 non-null   int64
8   Product Description Index 7991 non-null   int64
9   Order Quantity         7991 non-null   int64
10  Unit Selling Price     7991 non-null   float64
11  Unit Cost              7991 non-null   float64
dtypes: datetime64[ns](2), float64(2), int64(4), object(4)
```

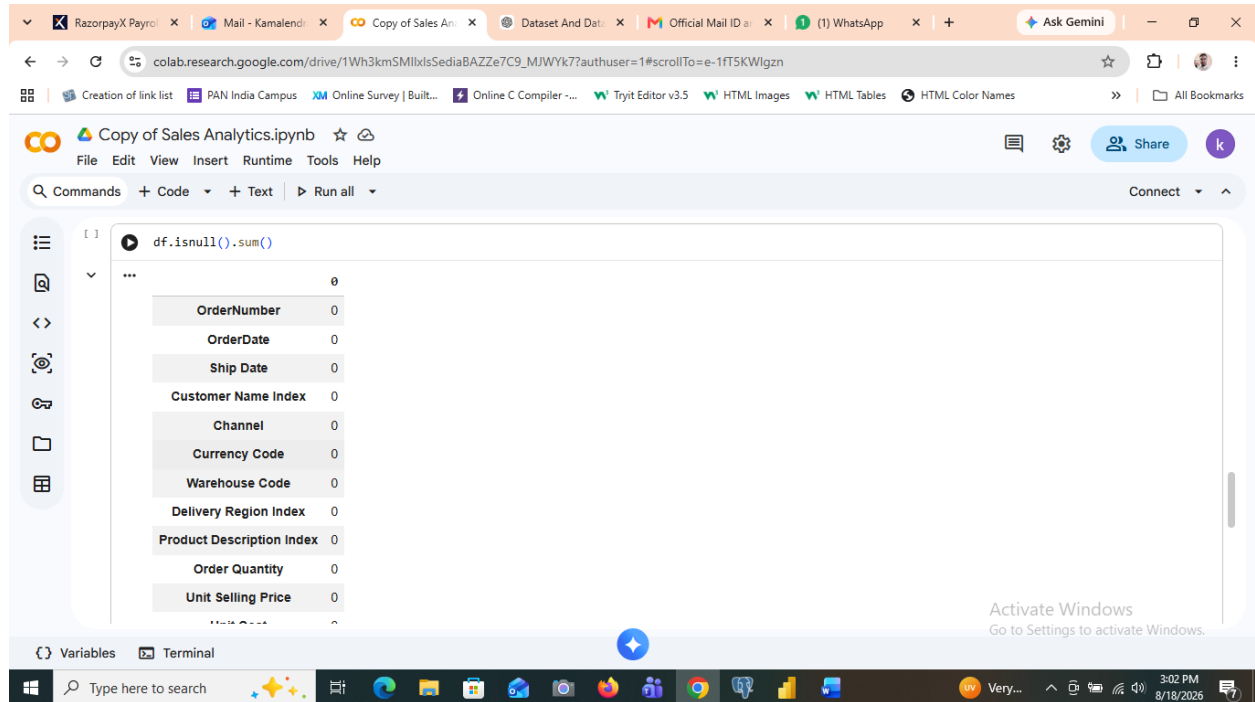
Check data types:

df.info()

```
df.shape
Out[1]: (7991, 12)

df.info()
Out[1]:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7991 entries, 0 to 7990
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  --
0   OrderNumber            7991 non-null   object
1   OrderDate              7991 non-null   datetime64[ns]
2   Ship Date              7991 non-null   datetime64[ns]
3   Customer Name Index    7991 non-null   int64
4   Channel                7991 non-null   object
5   Currency Code          7991 non-null   object
6   Warehouse Code         7991 non-null   object
7   Delivery Region Index  7991 non-null   int64
8   Product Description Index 7991 non-null   int64
9   Order Quantity         7991 non-null   int64
10  Unit Selling Price     7991 non-null   float64
11  Unit Cost              7991 non-null   float64
dtypes: datetime64[ns](2), float64(2), int64(4), object(4)
```

```
df.isnull().sum()
```



Total unit selling price:

```
df[["unit selling price"]].sum()
```

Download the cleaned CSV file from Colab

from google.colab import files

```
files.download("Cleaned_Sales_Dataset.csv")
```

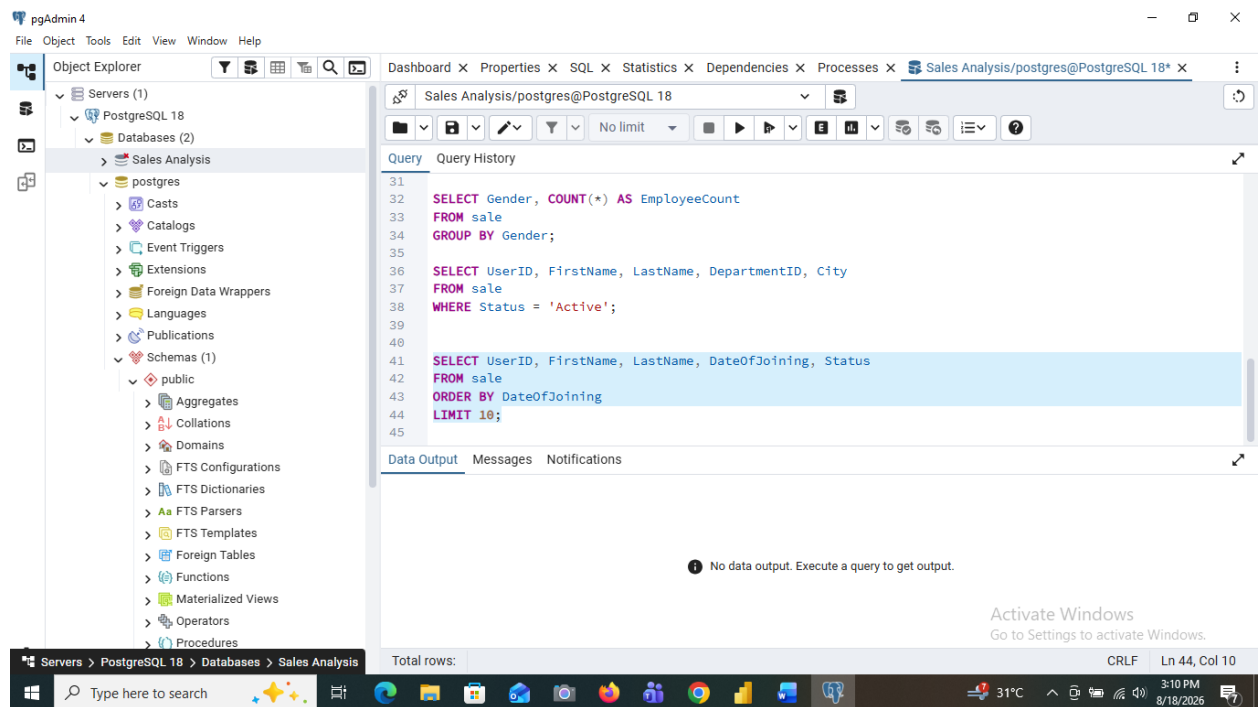
Step 2: Upload clean csv file into pgadmin data base

Create database in pgAdmin

Open pgAdmin → Servers → PostgreSQL → Databases

For example:

Database Name: sales analysis



2. Create the table

Query Tool create table and run:

CREATE TABLE Sale (

 UserID VARCHAR(20),

 EmployeeCode VARCHAR(20),

 FirstName VARCHAR(100),

 LastName VARCHAR(100),

 Gender VARCHAR(20),

 DepartmentID VARCHAR(20),

 RoleID VARCHAR(20),

 City VARCHAR(100),

 state VARCHAR(100),

 Country VARCHAR(100),

 Email VARCHAR(255),

Phone VARCHAR(20),
DateOfJoining DATE,
EmploymentType VARCHAR(50),
Status VARCHAR(50)
);

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree structure of the database: Servers (1) > PostgreSQL 18 > Databases (2) > Sales Analysis > postgres > public. The main pane shows a SQL query editor with the following code:

```
11 Country VARCHAR(100),  
12 Email VARCHAR(255),  
13 Phone VARCHAR(20),  
14 DateOfJoining DATE,  
15 EmploymentType VARCHAR(50),  
16 Status VARCHAR(50)  
17 );  
18  
19  
20  
21 SELECT *  
FROM sale;
```

Below the query editor, the 'Data Output' tab shows the results of the query. It displays a table with 7 rows and 6 columns: userid, employeecode, firstname, lastname, gender, and departmentid. The data is as follows:

userid	employeecode	firstname	lastname	gender	departmentid
U000001	EMP100001	Vivek	Rao	Male	D006
U000002	EMP100002	Priya	Gupta	Female	D008
U000003	EMP100003	Varun	Bose	Female	D009
U000004	EMP100004	Priya	Rao	Female	D002
U000005	EMP100005	Varun	Verma	Female	D002
U000006	EMP100006	Nisha	Bose	Male	D007
U000007	EMP100007	Sanjay	Joshi	Female	D007

The interface also shows a status bar at the bottom indicating 'Total rows: 999' and 'Query complete 00:00:07.186'. The Windows taskbar at the bottom shows the system time as 3:12 PM on 8/18/2026.

SELECT * FROM sale;

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with a tree view of the database structure. The main pane shows a SQL query window with the following query:

```
Country VARCHAR(100),
Email VARCHAR(255),
Phone VARCHAR(20),
DateOfJoining DATE,
EmploymentType VARCHAR(50),
Status VARCHAR(50)
);

SELECT *
FROM sale;
```

The 'Data Output' tab shows the results of the query, displaying 7 rows of data. The columns are: userid, employeecode, firstname, lastname, gender, and departmentid.

userid	employeecode	firstname	lastname	gender	departmentid
U000001	EMP100001	Vivek	Rao	Male	D006
U000002	EMP100002	Priya	Gupta	Female	D008
U000003	EMP100003	Varun	Bose	Female	D009
U000004	EMP100004	Priya	Rao	Female	D002
U000005	EMP100005	Varun	Verma	Female	D002
U000006	EMP100006	Nisha	Bose	Male	D007
U000007	EMP100007	Sanjay	Joshi	Female	D007

The status bar at the bottom indicates 'Total rows: 999' and 'Query complete 00:00:07.186'.

SELECT UserID, FirstName, LastName, City, Status
FROM sale
WHERE City = 'Chennai';

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with a tree view of the database structure. The main pane shows a SQL query window with the following query:

```
FROM sale;

SELECT UserID, FirstName, LastName, City, Status
FROM sale
WHERE City = 'Chennai';

SELECT Status, COUNT(*) AS EmployeeCount
FROM sale
GROUP BY Status
ORDER BY EmployeeCount DESC;
```

The 'Data Output' tab shows the results of the query, displaying 8 rows of data. The columns are: userid, firstname, lastname, city, and status.

userid	firstname	lastname	city	status
U000001	Vivek	Rao	Chennai	Resigned
U000013	Ajay	Das	Chennai	On Leave
U000024	Meera	Joshi	Chennai	On Leave
U000028	Divya	Gupta	Chennai	Active
U000041	Priya	Singh	Chennai	Inactive
U000048	Neha	Patel	Chennai	Resigned
U000051	Meera	Singh	Chennai	Inactive
U000053	Priya	Gupta	Chennai	Inactive

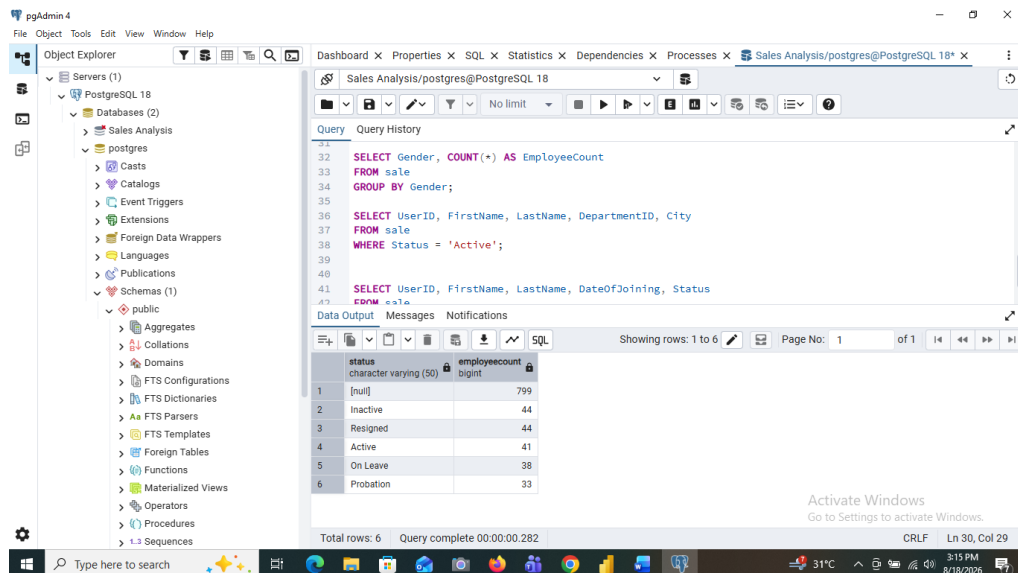
The status bar at the bottom indicates 'Total rows: 26' and 'Query complete 00:00:00.521'.

SELECT Status, COUNT(*) AS EmployeeCount

FROM sale

GROUP BY Status

ORDER BY Employee Count DESC;



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'public' schema. The main pane shows a SQL query window with the following query:

```

SELECT Gender, COUNT(*) AS EmployeeCount
FROM sale
GROUP BY Gender;

SELECT UserID, FirstName, LastName, DepartmentID, City
FROM sale
WHERE Status = 'Active';

SELECT UserID, FirstName, LastName, DateOfJoining, Status
FROM sale

```

The 'Data Output' tab shows the results of the first query, grouped by status:

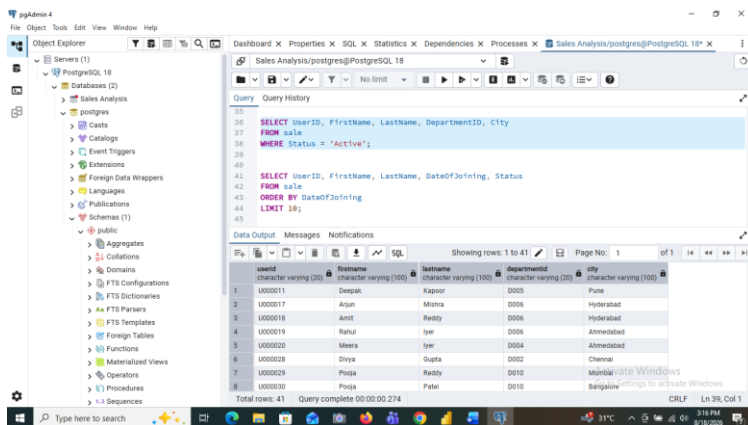
status	employeecount
1	799
2	44
3	44
4	41
5	38
6	33

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.282'.

SELECT UserID, FirstName, LastName, DepartmentID, City

FROM sale

WHERE Status = 'Active';



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure. The main pane shows a SQL query window with the following query:

```

SELECT UserID, FirstName, LastName, DepartmentID, City
FROM sale
WHERE Status = 'Active';

SELECT UserID, FirstName, LastName, DateOfJoining, Status
FROM sale
ORDER BY DateOfJoining
LIMIT 30;

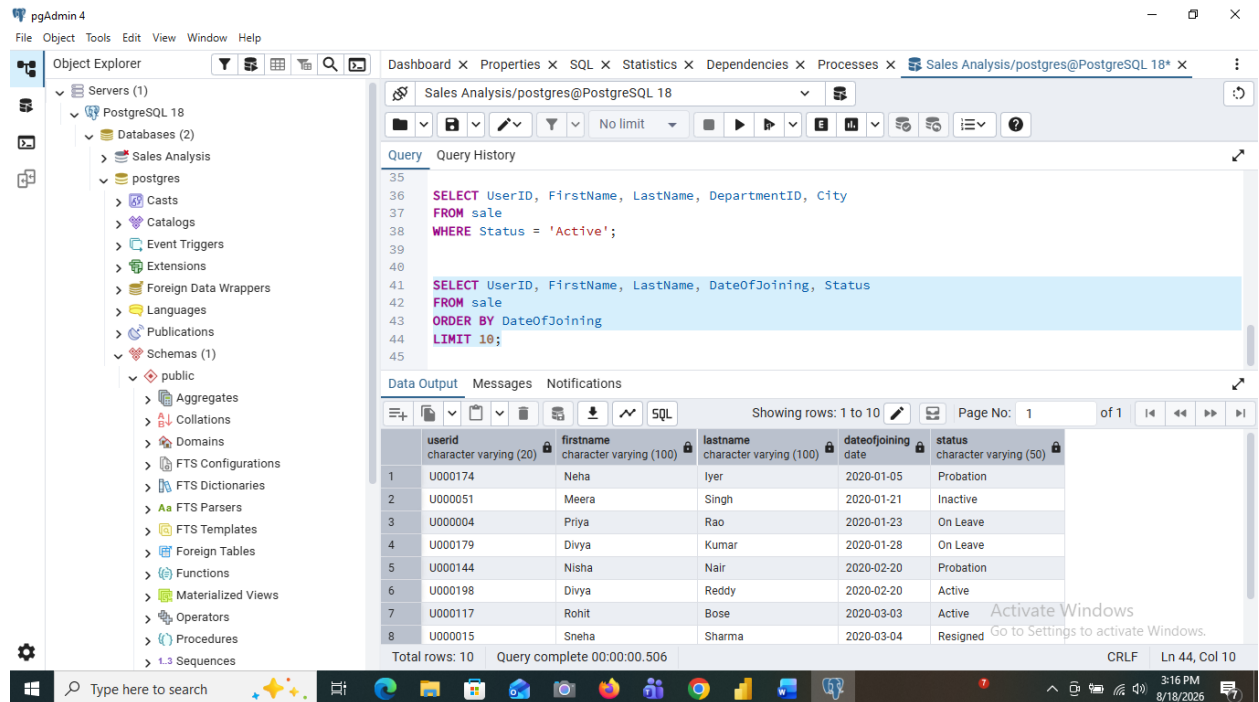
```

The 'Data Output' tab shows the results of the second query, ordered by DateOfJoining:

userid	firstname	lastname	departmentid	city
U000011	Dheepak	Kannan	0005	Pune
U000017	Arun	Mahesh	0005	Hyderabad
U000018	Amit	Reddy	0005	Hyderabad
U000019	Rahul	Iyer	0005	Ahmedabad
U000020	Meera	Iyer	0004	Ahmedabad
U000028	Divya	Gupta	0002	Chennai
U000029	Pooja	Reddy	0010	Mumbai
U000030	Pooja	Patel	0010	Bangalore

The status bar at the bottom indicates 'Total rows: 41' and 'Query complete 00:00:00.274'.

```
SELECT UserID, FirstName, LastName, DateOfJoining, Status  
  
FROM sale  
  
ORDER BY DateOfJoining  
  
LIMIT 10;
```



Do some required query in pgadmin after we completed

Move to powerbi for visualization

Step 3: powerbi (Dashboard)

Go to powerbi and click get data (sql)

Enter the connection

Server: localhost:5432

Database: sales_analysis

Enter PostgreSQL credentials

Username: postgres

Password:

Click **Connect**.

Select your table

sales_analysis

public

└─ sales data

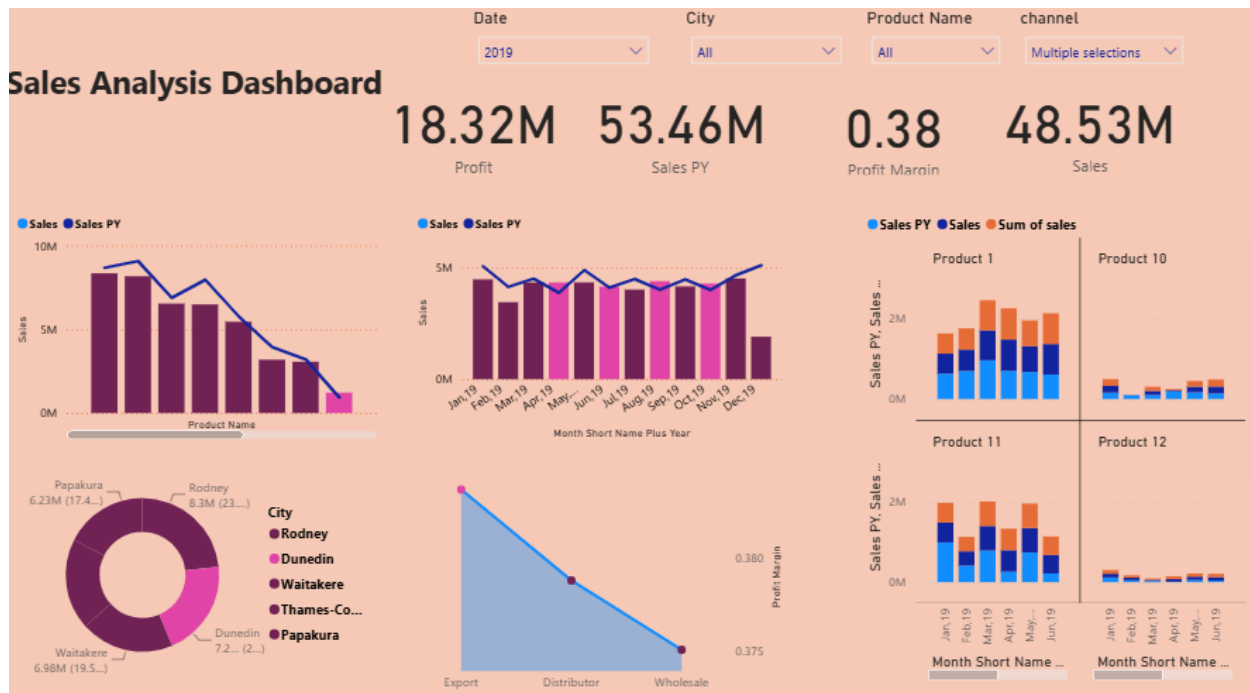
Select your table

Powerbi (visual):

Key Business Measures

Measure	Purpose
Sales	Measures revenue generated from sales transactions.
Sales PY	Compares current sales with the previous-year sales value.
Sales vs PY	Shows the variance or comparison between current sales and previous-year sales.
Profit	Measures sales value after deducting the associated cost.
Profit LY	Provides previous-year profit for historical comparison.
Profit Margin	Shows profitability relative to sales.

Dashboard Page 1 – Sales Dashboard



The first report page is designed as the primary management dashboard. It contains four slicers, five analytical charts, and four KPI cards. The slicers allow the user to dynamically filter the report.

Filters / Slicers

Visual	Field	Purpose
Date slicer	DateTable Year / Month-Year	Allows the user to analyze a selected year or month period.
City slicer	Regions_Table City	Filters the dashboard to a selected city.
Channel slicer	Sales_Data Channel	Filters sales by Distributor, Export, Wholesale, or other available channel values.

Product slicer	Products_Data Product Name	Filters the dashboard for a selected product.
----------------	----------------------------	---

Visual 1 – Sales by Product

This combo chart compares current Sales with Sales PY by Product Name and includes Sales vs PY as a comparison measure. It is useful for identifying products with strong growth, weak performance, or a decline compared with the previous year.

Primary dimension: Product Name.

Primary metric: Sales.

Comparison metrics: Sales PY and Sales vs PY.

Business use: product performance and year-over-year comparison.

Visual 2 – Monthly Sales Trend

This line and clustered-column combo chart compares monthly Sales with Sales PY. The Month-Year field is used as the time axis. It helps identify seasonality, growth patterns, and periods where sales are above or below the previous year.

Axis: Month-Year.

Columns/values: Sales and Sales PY.

Line/comparison: Sales vs PY.

Business use: monthly trend and year-over-year monitoring.

Visual 3 – Sales Distribution by City

The donut chart presents Sales by City and provides a quick view of the relative contribution of different cities. It is useful for understanding geographic concentration and identifying major sales markets.

Category: City.

Value: Sales.

Comparison/tooltip measure: Sales vs PY.

Business use: geographic contribution and market concentration.

Visual 4 – Profit by Channel

The area chart analyzes Profit by Channel and includes Profit LY and Profit Margin as supporting measures. It helps compare profitability across sales channels and understand whether higher sales are also producing stronger margins.

Category: Channel.

Primary metric: Profit.

Supporting metrics: Profit LY and Profit Margin.

Business use: channel profitability and margin evaluation.

Visual 5 – Customer Sales Performance

The clustered bar chart compares Sales and Sales PY across customer identifiers. It is intended to highlight high-value customers and customers whose sales have changed compared with the previous year.

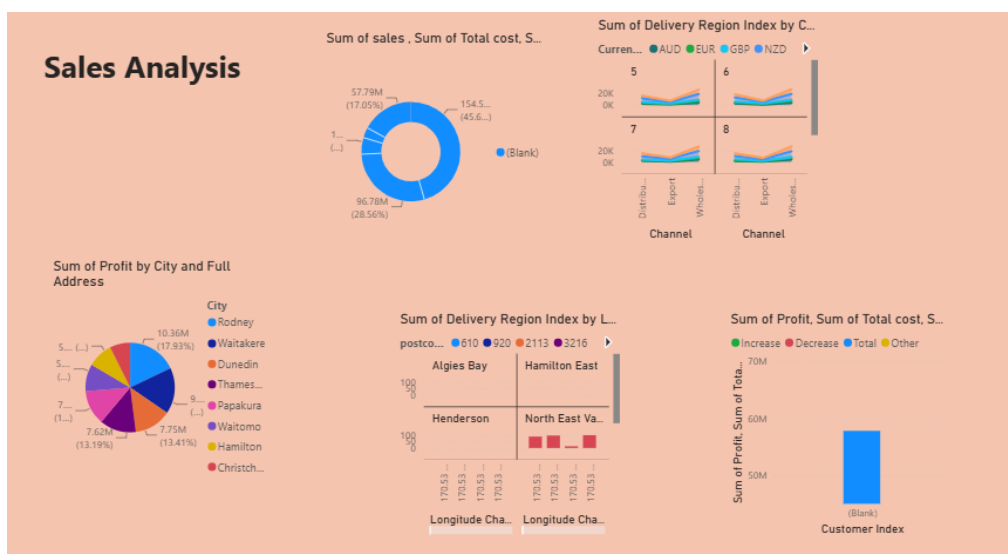
Category: Customer Name Index.

Values: Sales and Sales PY.

Comparison: Sales vs PY.

Business use: customer performance and account prioritization

Dashboard Page 2 – Sales Analytics



The second page provides deeper analytical views. It contains four visuals covering regional profitability, operational/transaction patterns, channel and customer-region patterns, and profit/cost movement.

Visual 1 – Regional Profit Analysis

The pie chart uses City as the geographic category and Profit as the value. Additional geographic fields such as Full Address, Longitude, Latitude, and Region Index are available to support location-based analysis and tooltips.

Category: City.

Value: Sum of Profit.

Supporting geographic fields: Address, Longitude, Latitude, Region Index.

Business use: identify profitable locations and geographic contribution.

Visual 2 – Regional / Operational Analysis

The line and stacked-column combination visual brings together geography, channel, order quantity, order date, customer, product, and delivery-region information. It can be used to explore how transaction volume and operational activity vary by geography and channel.

Geography: Longitude, postcode, suburb.

Business dimensions: Channel, Customer, Product.

Operational metric: Order Quantity.

Time field: Order Date.

Business use: regional and operational drill-down.

Visual 3 – Channel and Customer/Region Pattern

The stacked area chart compares Channel with aggregated delivery-region information and customer identifiers. Its purpose is exploratory: it helps users inspect how the selected channel and customer groupings are distributed across the available delivery-region values.

Category: Channel.

Supporting fields: Delivery Region Index and Customer Name Index.

Business use: exploratory channel/customer-region segmentation.

Visual 4 – Profit / Cost Waterfall Analysis

The waterfall visual is designed to examine profit movement together with customer, product, cost, sales, shipping date, unit price, unit cost, currency, channel, and delivery-region attributes. It is useful for explaining positive and negative contributions to profitability and for drilling into the drivers behind financial performance.

Primary financial metric: Profit.

Supporting financial fields: Total Cost, Sales, Unit Selling Price, Unit Cost.

Business dimensions: Customer, Product, Channel, Currency, Delivery Region.

Time/operations: Ship Date.

Business use: profit-driver analysis.

Technical Architecture

The recommended project architecture separates data preparation, database storage, and visualization.

Source: Excel → Preparation: Google Colab/Pandas → Storage: PostgreSQL → Database Administration: pgAdmin → Visualization: Power BI Desktop → Dashboard/Reporting.

Conclusion

The Sales Analysis project demonstrates the complete data analytics lifecycle: data ingestion, cleaning, transformation, database loading, analytical modeling, visualization, and business interpretation. The Power BI dashboard is designed to support both high-level KPI monitoring and detailed analysis. The combination of Google Colab, PostgreSQL/pgAdmin, and Power BI provides a practical end-to-end workflow suitable for a Data Analyst portfolio or project submission.